NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

MAJOR ASSIGNMENT

Course: Deep Learning

TOPIC: German Traffic Sign Recognition (GTSRB) From-Scratch CNNs versus Transfer Learning on 43 Classes

Instructor: Nguyễn Thị Kim Ngân

Students:
Nguyễn Tuấn Phong - 11230580
Trần Đình Tuấn Phong - 11230581
Đặng Nhật Huy - 11230544
Nguyễn Việt Hoàng - 11230539

Hanoi, October 2026

Part 1: Theory

Theoretical Background of Convolutional Neural Networks (CNNs)

A Convolutional Neural Network (CNN) is a neural network designed for grid-structured data such as images. Instead of connecting every input pixel to every neuron, a CNN scans the image with small learnable filters. This single idea, local connectivity with shared weights, is what makes CNNs both efficient and effective for vision, and it recurs in every model in this report.

1. Convolutional layer

A Convolutional Neural Network (CNN), on the other hand, is a specific type of neural network that can be used to process grid-like structured data like images. Rather than connecting each individual pixel in an image to all of the neurons in a layer, a CNN processes an image using many different, small filters that are learned during training. The use of locally connected neurons with weight sharing allows CNNs to be computationally efficient while being highly effective at processing images. These two aspects of CNNs have recurred throughout each of the models discussed in this report.

2. Pooling layer

Pooling (max-pooling with usually 2×2 stride) gets a summary of how each point of a feature map relates to local surrounding by choosing the response of the neuron which is the strongest one in that neighborhood. It can lead to a reduction in computation by making the spatial size of the feature maps smaller. It can allow the network to be a little forgiving of small translations. Convolution and pooling stacked together make sure that each deeper layer of the network is actually a little larger receptive field on a feature map, a feature map of a lower network level.

3. Fully-connected layer and the parameter problem

Feature maps at the output of a classic CNN are typically flattened into one-dimensional vectors that are fed through fully-connected layers, also known as dense layers, to get the class scores.

This is an area where the use of parameters becomes very costly: flattening a 64×12×12 feature map gives a vector that has 9,216 elements, connecting each of these elements to 256 neurons in a layer would require around 2.36 million weights.

In our baseline model, this is the only layer that has 97.3% of all parameters, which makes us realize why in current architectures large dense heads are usually replaced by other elements.

4. Techniques that make deep CNNs trainable

The classic CNN ends up flattening feature maps at the end into a vector and then passing this one through dense (fully-connected) layers in order to classify them. Here, the amount of parameters explodes really drastically: Flattening a single 64×12×12 feature map results in a total of 9,216 features. A connection of these to 256 neural units will cost about 2.36 million of weights in one single layer. Actually, that layer alone, our baseline model's, accounts for 97.3% of the total number of parameters. Hence modern architectures are very wary of employing large dense heads.

- **Batch Normalization:** it normalizes each channel in the mini-batch, then rescales it with two learnable parameters. It effectively flattens the loss surface while allowing the use of higher learning rates, as well as more stable and fast learning.
- **Residual connections (y = F(x) + x):** it adds the output of a block to its input. The '+1' in the gradient serves as a shortcut, so the gradients can be propagated in the reverse direction, and thus degradation is overcome and training of very deep networks becomes possible. When the channels are different the shortcut is projected to the proper size via 1x1 convolution.
- **Dropout and Spatial Dropout:** These methods randomly "kill" neurons during learning so that no single feature can become too important. It turns out that ordinary dropout is not particularly helpful on feature maps where nearby pixels are related. On the other hand, Spatial Dropout removes entire channels (feature detectors) and as such works as a regularization method to a higher extent.
- **Global Average Pooling (GAP):** it simply averages each channel to a single value, thus it replaces flatten + dense. The very dense head, a heavy parameters one, is being removed, and this is acting as a sort of structural regularization. In our complex model it reduces the head parameters from 2.36 M to about 11 thousand, a 214× reduction.

5. Transfer learning

Deep CNNs require a lot of data to be trained effectively. In transfer learning, you reuse a network that's already been trained using a substantial dataset (like ImageNet containing 1.28 million images). The basic idea is that since the early convolutional layers will learn features common to lots of natural images, only the last few layers that are task-specific will need to be replaced and retrained. During fine-tuning you use discriminative learning rates (learning rate starts small for early layers and then gets large for the new head). Also freeze-then-unfreeze training is being used to prevent large initial gradients from the randomly-initialized head from destroying the good pre-trained filters.

Part 2: Practical Application

I. Problem Description

Title: German Traffic Sign Recognition: comparing a from-scratch simple CNN, a from-scratch complex CNN, and transfer learning, on the GTSRB benchmark (43 classes).

Research purposes:
(1) Develop and evaluate three deep-learning models for traffic-sign recognition.
(2) Benchmark a lightweight custom CNN against pre-trained ImageNet models.
(3) Measure model performance on corrupted images and deployment speed on embedded hardware.

Our Input: A cut-out of a colour traffic sign picture. The size differs every time. The model brings it to a fixed dimension.

Our Output: The corresponding class is one out of the 43 traffic sign classes (speed limits, prohibitions, warnings, mandatory directions, etc.).

Overview of work done:
- Data gathering and indexing.
- Track-aware preprocessing and splitting to prevent leakage.
- Development and training of three model variants, namely, M1, M2 and M3.
- Performance evaluation of models based on accuracy, macro-F1, class-wise performance, McNemar tests, calibration (ECE), five corrupted versions, and CPU latency, as well as a leakage control experiment and a grad-CAM explanatory figure.

II. Dataset Description

- **Download the dataset** from: https://sid.erda.dk/public/archives/daaeac0d7ce1152aea9b61d9f1e19370
- Or from here: kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign
- **Size and content:** The GTSRB dataset has about 51 thousand color pictures of different road signs that car owner can see while he is moving. The dataset comprises 39 thousand plus training images and 12 thousand plus testing samples over 43 different classes.
- **Image size:** The sizes of images are not fixed. In terms of size, the median is about 43x43, and ranges from 25x25 to 266x266. Since these signs are generally photographed at a distance, the details such as speed limit numbers become difficult to see.
- **Data sequence:** Each road sign was recorded by a moving camera for a while in which you get a continuous sequence of 30 pictures (a so-called "track"). The pictures

of a particular track are very similar, which is something to be aware of when performing the data split, as explained in the note III.

- **Class imbalance:** Maximum one has 2250 training samples. Minimum one has 210 samples. And this imbalance is 10.7:1. Accuracy score is not the major performance indicator as Macro-F1 is used instead.

III. CNN Model Design

1. Data preprocessing (clean, normalize, augment)

- Crop: crop the sign region of interest including 10% margin. Remove the most part of the background while preserving the red/blue border.
- Contrast equalization (CLAHE): apply it to the L (lightness) channel of LAB colour space. Fix the lighting without altering the colour.
- Resize: 48×48 for the scratch models (M1, M2) Transfer models (M3) resize images to 224×224 so that the size matches the input scale of the pretrained networks.
- Normalization: channel-wise standardization. For M1/M2 use training statistics from GTSRB. Pretrained ImageNet statistics for M3.
- Augmentation: rotation ±15°, translation ±10%, zoom 0.9 to 1.1, brightness/contrast jitter. Do not perform horizontal flip (19↔20, 33↔34, 36↔37, 38↔39 pair of classes are mirror images). Hue/Saturation modification is strictly off since colour is not merely for fun.



2. Dataset split (track-aware, three subsets)

The official test set contains 12,630 images which are sealed permanently. 39,209 training images are divided into an 80/20 ratio using the class-id and track-id group key, as a result of StratifiedGroupKFold. This division ensures that all 30 frames of a physical object remain in one partition. On the other hand, a random per-image split method could lead to the same nearly identical frames appearing both in training and validation sets.

It is important to prevent the leakage of near-duplicate frames from one set to the other, that is why a random per-image split would result in such leakage. To make sure that no shared tracks are leaked and that frames that are similar but not exact duplicates (near-duplicates) would not be present in both sets, an automatic assertion is used to verify zero shared tracks between the two sets.

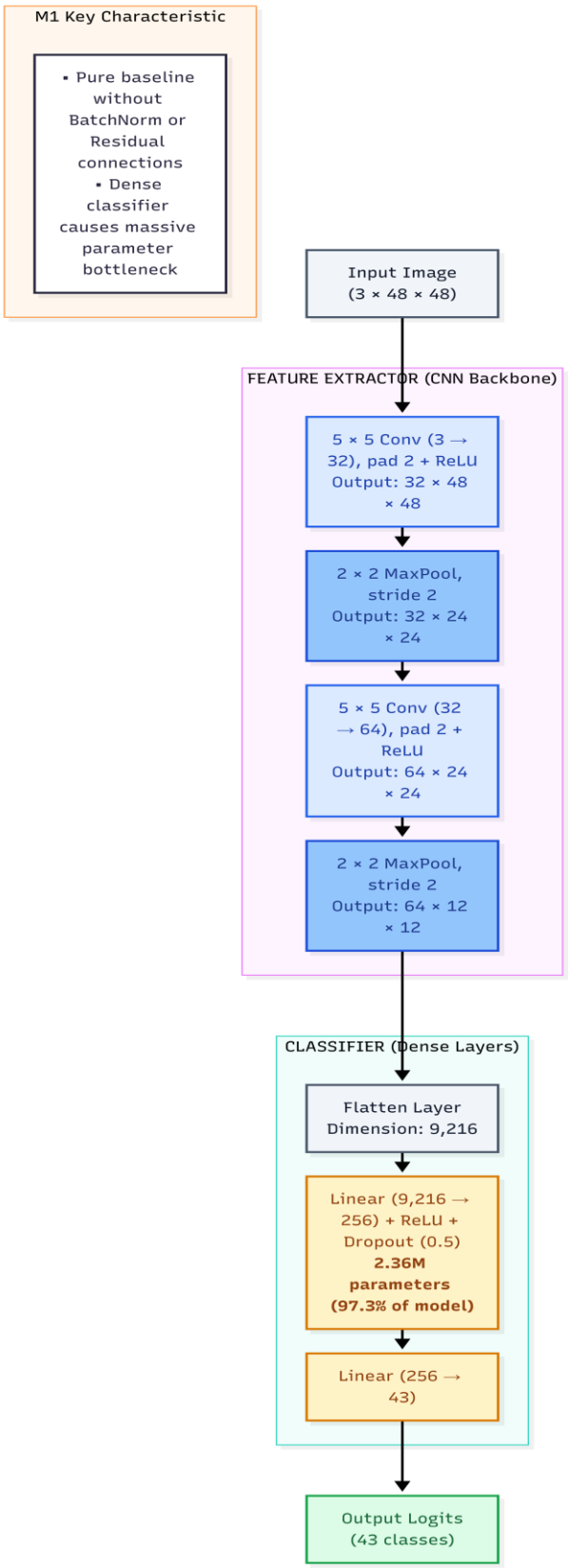
Subset	Number of samples	Purpose
Training	31,379	Model Training
Validation	7,830	Tuning
Test (official)	12,630	Evaluating model performance

3. Simple CNN built from scratch (M1, LeNet-style)

Layer	Output shape	Notes
Input	3 × 48 × 48	RGB image
Conv 5×5 (3→32) + ReLU + MaxPool2	32 × 24 × 24	block 1
Conv 5×5 (32→64) + ReLU + MaxPool2	64 × 12 × 12	block 2
Flatten	9,216	n/a
Linear 9216→256 + ReLU + Dropout	256	97.3% of all params
Linear 256→43	43	class scores

Total parameters: 2.42 M.

Two convolution with pooling blocks had by two dense layers. It omits BatchNorm, residuals and spatial dropout then it is a good baseline.



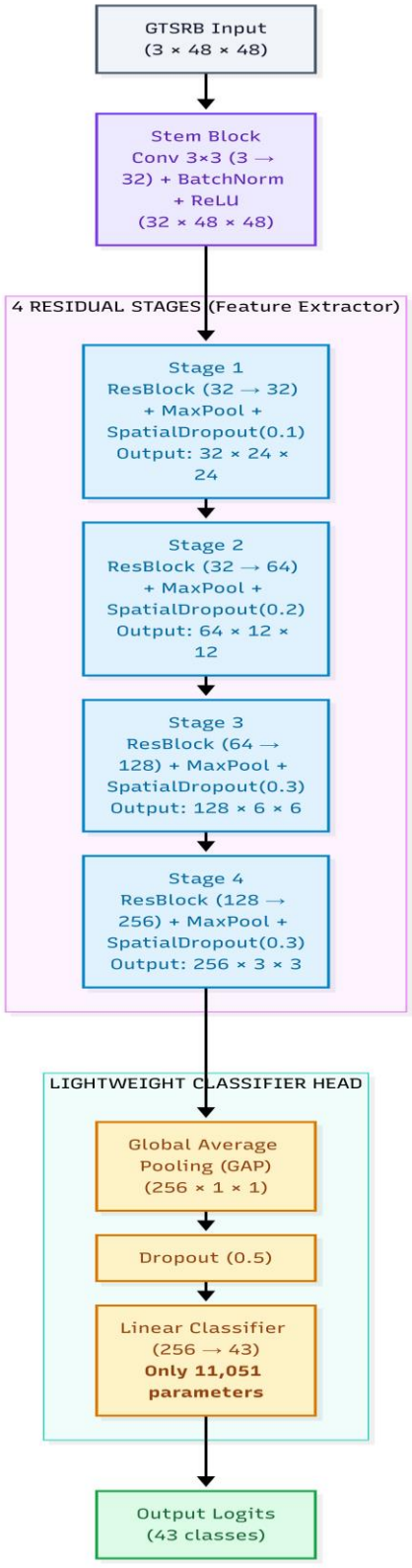
M1: LeNet-5 architecture

4. Complex CNN built from scratch (M2, VGG-style with residual blocks)

Stage	Output shape	Components
Stem: Conv 3×3 (3→32) + BN + ReLU	32 × 48 × 48	n/a
Stage 1: ResBlock(32→32) + Pool + SDrop	32 × 24 × 24	spatial dropout 0.1
Stage 2: ResBlock(32→64) + Pool + SDrop	64 × 12 × 12	spatial dropout 0.2
Stage 3: ResBlock(64→128) + Pool + SDrop	128 × 6 × 6	spatial dropout 0.3
Stage 4: ResBlock(128→256) + Pool + SDrop	256 × 3 × 3	spatial dropout 0.3
Head: GAP → Dropout → Linear 256→43	43	head ~11 K params

Total parameters: 1.24 M. Each ResBlock = two 3×3 convolutions + BatchNorm with a skip connection.

Four stages with BatchNorm, residual connections, spatial dropout, and a Global Average Pooling head. Despite more convolutional layers than M1, it has fewer than half the parameters, because GAP replaces the costly dense head.

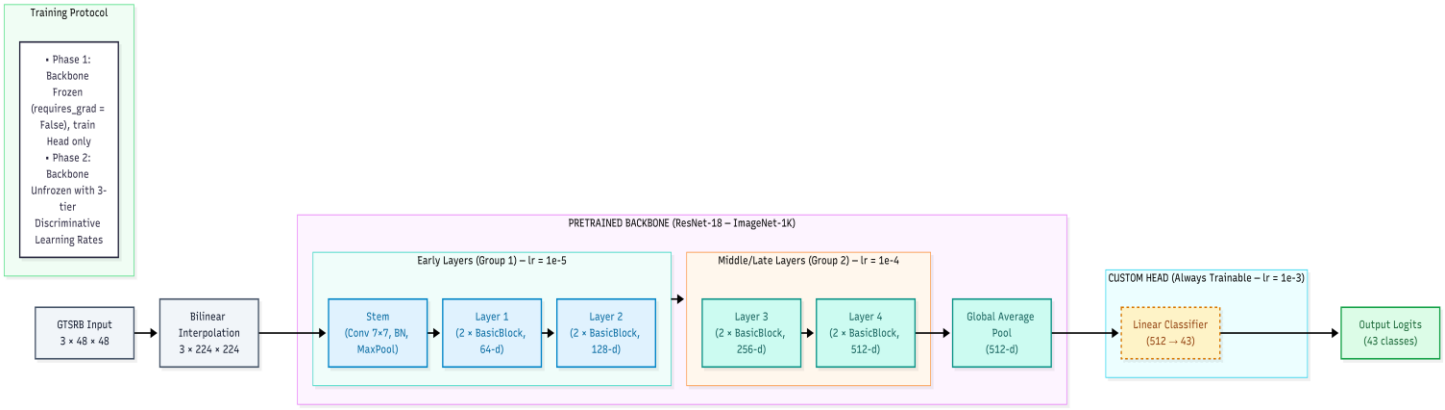


M2: VGG-Res architecture

5. Transfer-learning model (M3, pretrained ImageNet backbones)

Component	Detail
Backbone	ResNet18 / MobileNetV2 / EfficientNet-B0 (ImageNet-pretrained)
Input	224 × 224, ImageNet normalization
Head	backbone final layer replaced by Linear → 43 classes
Fine-tuning	Phase 1 freeze head-only, Phase 2 unfreeze + discriminative LR (AdamW)

M3 adopts a similar approach with ImageNet-trained models, only here, the head of the network is transformed to handle 43 classes instead of 1000. There's a three-way comparison of the backbone architecture ResNet18 (network with dense residual convolutions), MobileNetV2 (depthwise-separable convolutions with inverted residuals), and EfficientNet-B0 (compound scaling combined with squeeze-and-excitation). All image inputs are resized to 224×224 (the backbones reduce images 32× to 224 for the final 7×7 feature map). The M3 model first is trained only a head (3 epochs) while the rest are kept fixed. Afterwards, it is completely unfixed and training continues with different learning rates, using early, middle and head parts at different learning rates from 1e-5 to 1e-3 via the AdamW optimizer.



For MobileNetV2 and EfficientNet-B0, the backbone is replaced by inverted residual blocks with a 1280-dimensional feature output and a Dropout(0.2) + Linear(1280, 43) head.

IV. Experimental Results

All models are evaluated on the sealed official test set of 12,630 images. Macro-F1 is the primary metric because of the 10.7:1 class imbalance; accuracy, macro-precision and macro-recall are reported alongside.

Model	Accuracy	Precision	Recall	F1-score
M1 LeNet (simple, scratch)	98.46%	0.9796	0.9770	0.9778
M2 VGG-Res (complex, scratch)	99.26%	0.9888	0.9879	0.9880
M3 ResNet18 (transfer)	99.33%	0.9901	0.9911	0.9903
M3 MobileNetV2 (transfer)	98.73%	0.9833	0.9779	0.9796
M3 EfficientNet-B0 (transfer)	98.76%	0.9847	0.9839	0.9838

Precision, recall and F1 are macro-averaged over all 43 classes (every class weighted equally).

Statistical significance (McNemar test)

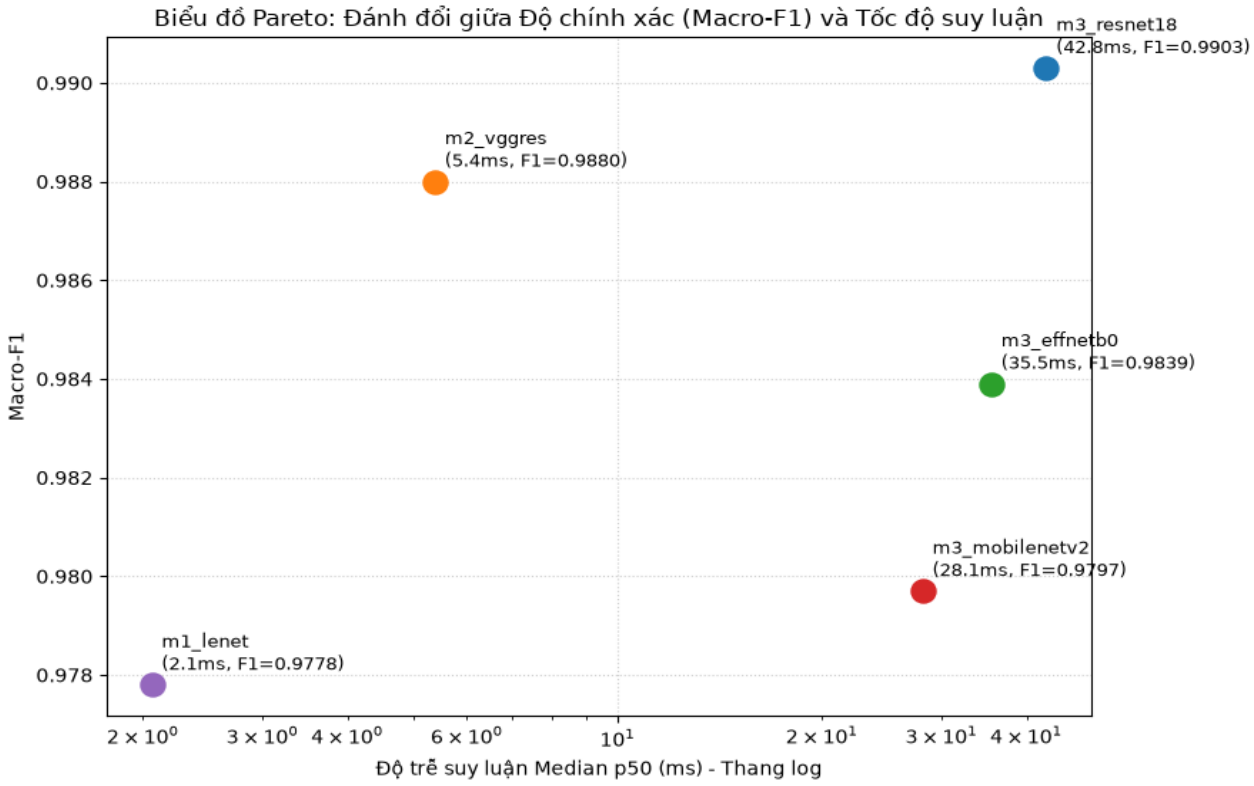
If all models are tested on the same images, this will pair the observations. Therefore, McNemar's test is the right one (although a two-sample t-test is the wrong statistical way). The main things: the handcrafted M2 outperforms EfficientNet-B0 and MobileNetV2 respectively ($p = 6.3e-07$; $3.9e-07$) as statistically proven. Besides, it is no different compared to ResNet18 (statistical equivalence, $p = 0.448$). Hence the small model of about 1.24 M-parameters made from scratch equals the big model with nearly 11.2 M-parameters pretrained ones.

Calibration, robustness and speed

- **Calibration (Expected Calibration Error):** M2 is best calibrated (ECE 0.053), about half the error of the other models which make its predicted confidences most reliable.
- **Robustness (mean corruption error over 5 corruptions, lower is better):** M1 0.259, M3-ResNet18 0.282, M2 0.288, MobileNetV2 0.523, EfficientNet-B0 0.538. The most inaccurate one (M1) turns out to be the most reliable one to the point of making you wonder if it isn't always clean accuracy that makes the best robustness.
- **Inference speed (CPU, per image):** M1 0.9 ms, M2 3.2 ms, ResNet18 16.6 ms, MobileNetV2 47.9 ms, EfficientNet-B0 243.4 ms. Latency does not correlate with FLOPs: in this case, 'EfficientNet' is actually the slowest model.

Overall comparison

M2 (the self-built complex CNN) offers the best overall trade-off: accuracy statistically tied with the best transfer model, the best calibration, 9× fewer parameters than ResNet18, and about 5× faster inference. Transfer learning gives the single highest score (ResNet18, 0.9903 macro-F1) but at a large parameter and latency cost, without a statistically significant advantage over M2.



V. Conclusion

Our team built and compared three model in GTSRB traffic-sign classification problem. The main findings are:

- A CNN built from scratch (M2) matches ImageNet-pretrained model: statistically equivalent to ResNet18 (McNemar p-value = 0.448) and better than MobileNetV2 and EfficientNet-B0 (p-value < 1e-6), while being smaller and faster.
- Inference latency does not follow FLOPs: EfficientNet-B0 is the slowest model.
- The most accurate model is not the most robust: M1 has the lowest clean accuracy but the best robustness.
- Data splitting: a random split inflates validation performance by about 1.09 pnts because of data leakage.

- The champion is M2: top-tier accuracy, smallest size, and better inference to deploy.

References

[1] J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel. The German Traffic Sign Recognition Benchmark: A multi-class classification competition. IJCNN, 2011.

[2] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. Proc. IEEE, 1998. (LeNet)

[3] K. Simonyan, A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR, 2015. (VGG)

[4] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016. (ResNet)

[5] M. Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR, 2018.

[6] M. Tan, Q. Le. EfficientNet: Rethinking Model Scaling for CNNs. ICML, 2019.

[7] S. Ioffe, C. Szegedy. Batch Normalization. ICML, 2015.

[8] R. Selvaraju et al. Grad-CAM: Visual Explanations from Deep Networks. ICCV, 2017.